Test Redirection

Our first test will be for the request GET /messages. We want to ensure that the request is redirected based on whether there's an existing conversation or not. Add the following describe block to your test module:

phone_app/test/phone_app_web/controllers/message_controller_test.exs

```
describe "GET /messages" do
  test "empty messages redirects to new message", %{conn: conn} do
    conn = get(conn, ~p"/messages")
    assert redirected_to(conn, 302) == "/messages/new"
  end

  test "redirects to latest messages", %{conn: conn} do
    _m1 = SmsMessageFactory.create(%{to: "111-222-3333", body: "Test 1"})
    m2 = SmsMessageFactory.create(%{to: "211-222-3333", body: "Test 2"})

    conn = get(conn, ~p"/messages")
    assert redirected_to(conn, 302) == "/messages/#{m2.contact_id}"
  end
end
```

The second argument of each test block—%{conn: conn}—takes the conn set up by ConnCase and injects it into the test block. We pass the conn variable into the get/2 function along with the route that we want to run against. The endpoint is then executed, and the conn variable is transformed based on the output of the request.

In this test, we use the redirected_to/2 function to assert that our requests are correctly redirected. We use our SMS factory to set up the data used in our test.

Test HTML Responses

Our next test will be for the request GET /messages/new. This controller returns HTML containing a form, so we want to make sure that's being returned correctly. Add the following describe block to your test module:

phone_app/test/phone_app_web/controllers/message_controller_test.exs

```
describe "GET /messages/new" do
  test "a message form is rendered", %{conn: conn} do
    conn = get(conn, ~p"/messages/new")

    assert html = html_response(conn, 200)
    assert html =~ ~S(<form action="/messages/new" method="post")
    assert html =~ "Send a message..."
    assert html =~ "To (Phone Number)"
  end
end
```